



From Classical to Quantum Computing: Working Together on Grand Challenges

Ezhilmathi Krishnasamy

Rudolfovo – Science and Technology Center Novo Mesto

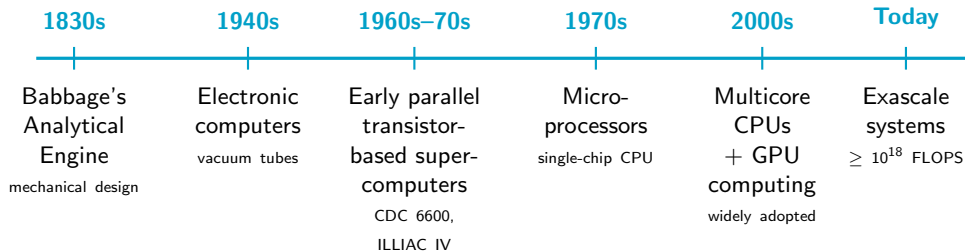
July 9, 2026



{ From Mechanical Ideas to Exascale Computing



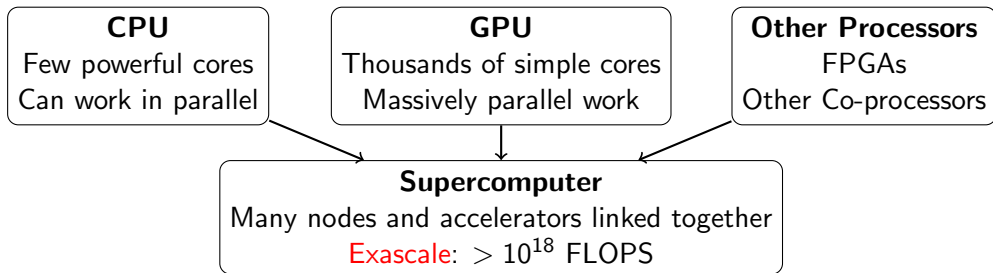
Computation evolved from mechanical designs into electronic machines capable of performing more than 10^{18} floating point operations per second.



The goal remained the same: automating computation. What changed was the technology: from mechanical motion to electronic circuits, early parallel machines, microprocessors, accelerators, and exascale supercomputers.



Present Scenario



EuroHPC JU: Our Supercomputers

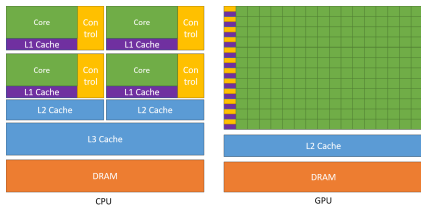
EuroHPC JU supercomputers: Overview of EuroHPC systems, locations, and capabilities.

TOP500

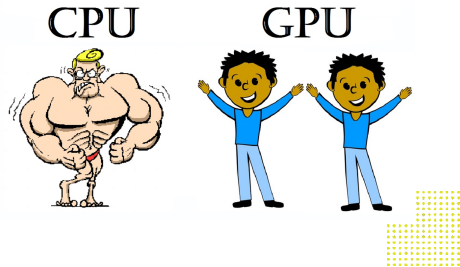
TOP500 ranking: Global ranking of the world's most powerful supercomputers.

Important differences between CPU and GPU

- GPU has many cores compared to CPU
- But on the other hand, the CPU's frequency is higher than the GPU. That makes the CPU faster per core than the GPU.
 - Intel® Core™ i7-10700K Processor base frequency is **3.80 GHz**, whereas, Nvidia Ampere has **0.765 GHz**



Source: [Nvidia: CUDA programming](#)



{ Supercomputers from Europe



Figure: BSC-Barcelona



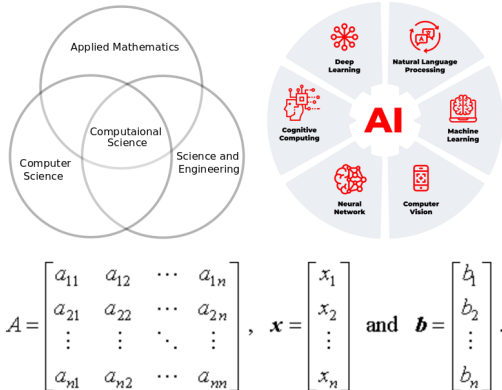
Figure: JUPITER-Jülich, Germany



Figure: LUMI-Finland



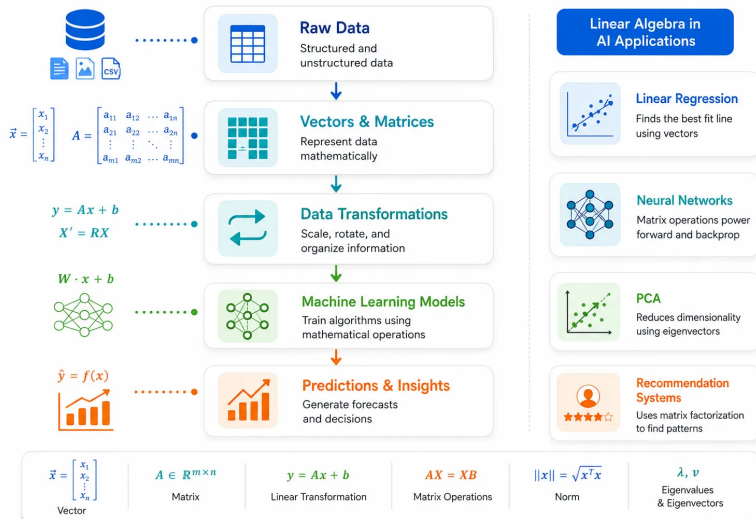
Scientific Computing - System of Linear Equations



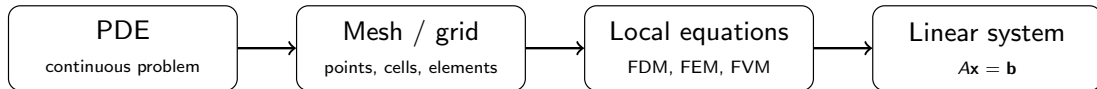
- Many problems in science and engineering are described by differential equations, especially partial differential equations (PDEs).
- These PDEs describe how physical quantities change in space and time.
 - Examples include heat transfer, fluid flow, wave propagation, weather forecasting, and structural mechanics.
- To solve PDEs on a computer, the continuous physical domain must be discretized into a finite number of points, cells, or elements.
- These systems are then solved using direct or iterative solvers to determine the unknown variables.
- Similarly, many artificial intelligence applications involve large matrix and vector computations.



Why AI and ML Need Linear Algebra



Science and Engineering: Discretization Leads to Linear Systems



Finite difference example

$$-\frac{d^2 u}{dx^2} = f(x) \quad \Rightarrow \quad \frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i$$

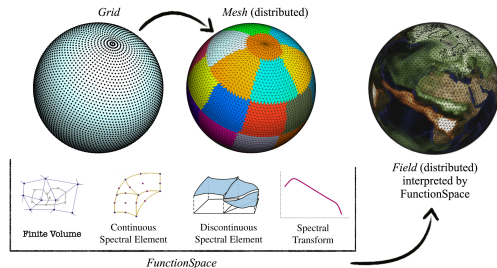
$$\underbrace{\frac{1}{h^2} \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}}_A: \text{coefficients from the stencil} \underbrace{\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}}_x: \text{unknown values} = \underbrace{\begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}}_b: \text{known data}$$

Each grid point gives one equation; each equation becomes one row of A .



Applications and Examples

- In weather forecasting, the domain size is discretized by numerical methods. For example, 786 thousand cores from the IBM Blue Gene/Q Mira system at Argonne National Laboratory are needed to simulate a resolution of 3 km (1.8 billion cells) using spectral element methods (one kind of numerical method)¹.



Mesh and simulation settings ²

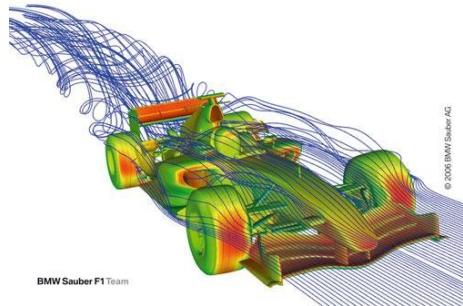
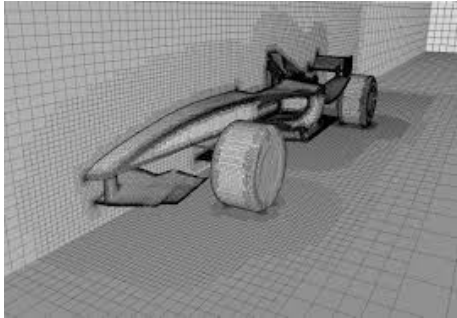
¹ HPC for Weather Forecasting, John Michalakes

² A library for numerical weather prediction and climate modeling, Willem Deconinck, et al.



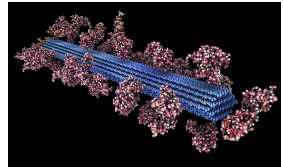
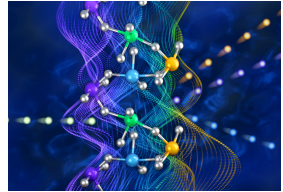
{ Applications and Examples

- **CERFACS** in France used up to 250 000 cores with grids of 2 to 4 billion cells (using Large Eddy Simulation - one kind of numerical methods strategy) for computational fluid dynamics application.



{ Applications and Examples

- **Materials science:** design new materials with desired properties.
- **Biology:** understand proteins and support drug discovery.
- **Engineering:** simulate fluids, heat transfer, and structures to improve designs.



{ Limitations of Classical Computers

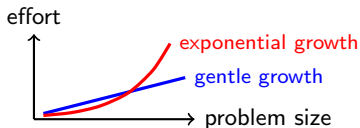


- Classical computers and supercomputers are powerful tools for scientific computing, especially for solving PDEs with numerical methods.
- However, CPU performance is increasingly limited by three walls:
 - **Memory wall:** data movement is slower than computation.
 - **Power wall:** higher frequency increases heat and power consumption.
 - **ILP wall:** instruction dependencies limit parallel execution.
- These limits motivate alternative and complementary computing approaches, such as GPUs, accelerators, and **QUANTUM COMPUTING**.



{ Some Problems Push Back

Some problems grow **exponentially** harder as they get bigger. More processors alone are not enough.



Example: quantum systems

Simulating molecules is difficult because the number of possible quantum states can grow exponentially with system size.

- **Quantum chemistry:** molecular states can grow exponentially.
- **Drug discovery:** many possible molecule–protein interactions.
- **Materials science:** electronic states grow rapidly with system size.
- **Optimization:** possible solutions can grow exponentially.



{ A natural idea



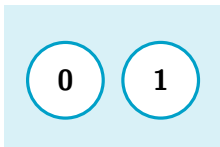
If nature itself is **quantum**,
perhaps a **quantum** machine is the natural tool to reason about it.

(an idea often credited to Richard Feynman, 1982)



{ Bit versus qubit

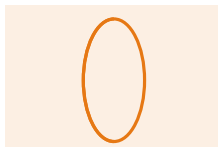
Classical bit



exactly one value

A coin lying **flat**: heads or tails.

Quantum qubit



0 and 1 together

A coin **spinning**: both at once, until it lands.



{ The spinning-coin analogy



- A spinning coin is neither heads nor tails — a **blend** of both. (superposition)
- **1 coin** $\rightarrow$ 2 states. **2** $\rightarrow$ 4. **10** $\rightarrow$ 1,024. $n \rightarrow 2^n$.
- Qubits can be **linked** so their outcomes correlate. (entanglement — Einstein's “spooky action”)

The promise

A modest number of qubits represents an **enormous** space of possibilities at once.



{ But there is a catch

Take-away

Quantum computers are **not** just faster classical computers. They compute in a **fundamentally different way** — powerful for some problems, irrelevant for most.

When you **measure**, the spinning coin lands on one answer.

- We cannot read out all 2^n possibilities.
- A quantum algorithm arranges interference so the right answer is most likely to appear.

Quantum Computing	Vs.	Classical Computing
 Calculates with qubits, which can represent 0 and 1 at the same time		 Calculates with transistors, which can represent either 0 or 1
 Power increases exponentially in proportion to the number of qubits		 Power increases in a 1:1 relationship with the number of transistors
 Quantum computers have high error rates and need to be kept ultracold		 Classical computers have low error rates and can operate at room temp
 Well suited for tasks like optimization problems, data analysis, and simulations		 Most everyday processing is best handled by classical computers

{ The hardware, in one sentence

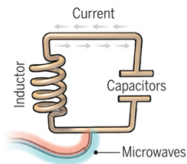
Many ways to build qubits — superconducting circuits, trapped ions, neutral atoms, photons —

but **all** face the same challenge:

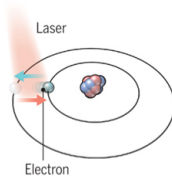
keeping quantum information stable and reducing errors.

A bit of the action

In the race to build a quantum computer, companies are pursuing many types of quantum bits, or qubits, each with its own strengths and weaknesses.



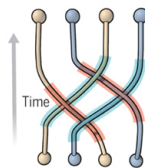
Superconducting loops



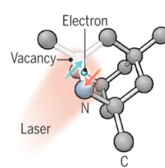
Trapped ions



Silicon quantum dots



Topological qubits



Diamond vacancies

{ Noise, errors, and the early stage



Decoherence

A spinning coin is fragile: heat, vibration, or stray fields make it “fall over” and lose its state.

Errors

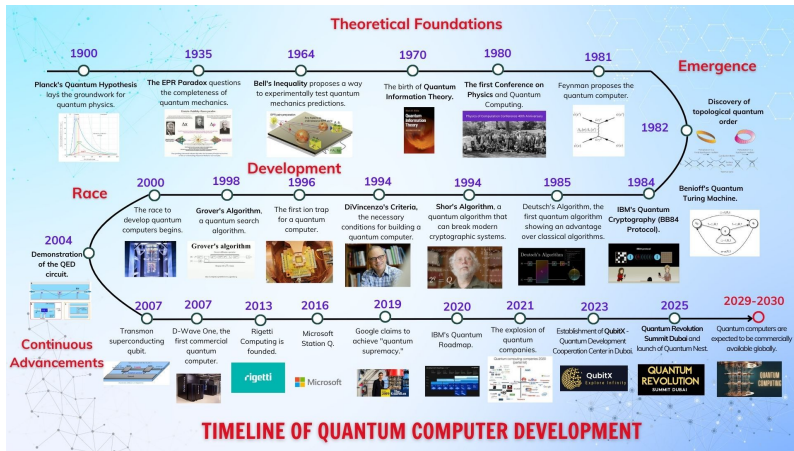
Operations are imperfect; errors accumulate, limiting how long a computation can run.

- Today's machines are **noisy** and **small** — the “NISQ” era.
- **Error correction** spreads one reliable “logical” qubit across many physical ones.
- Works in principle — but needs far more, better qubits than we have.



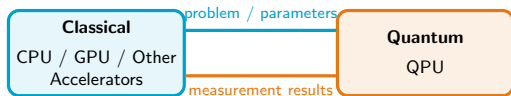
An honest status check

Quantum computing is **real and progressing fast**, but **still early**.

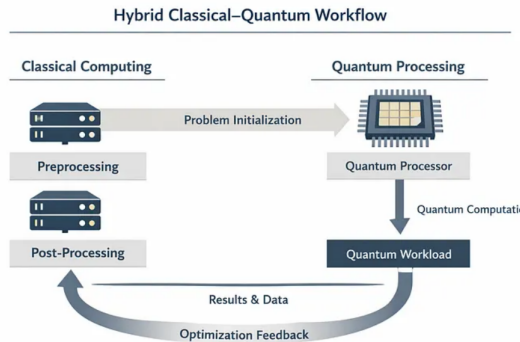


{ Not a replacement — a partnership

Quantum processors are specialists. They sit **alongside** classical computing (CPUs), each doing what it does best.



The two loop back and forth until the problem is solved.



{ Application 1 — Materials & drug discovery



- Molecules and materials are **governed by quantum mechanics**.
- Classical simulation hits a wall for strongly interacting electrons.
- Quantum hardware could model molecules **natively**.

Could mean: better batteries, catalysts, fertilizers; faster drug screening.

The hybrid split

Quantum: the electronic-structure core.

Classical: geometry, sampling, ranking candidates.



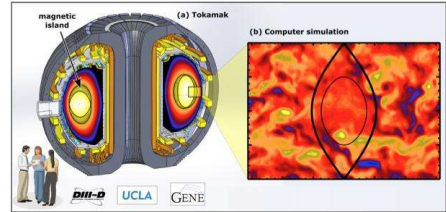
{ Application 2 — Physics simulations

- Plasma simulation: involves turbulence, which requires enormous computational power.
- Quantum routines could accelerate **selected sub-problems**, not whole simulations.

The hybrid split

Classical HPC: the full simulation and visualization.

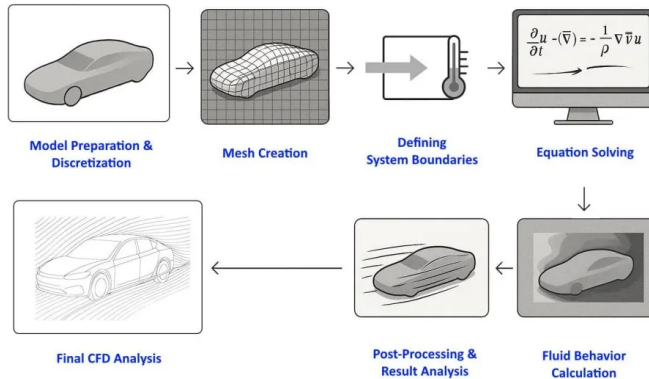
Quantum: targeted kernels that are bottlenecks classically.



Plasma Simulation.



Application 2 — Physics simulations



- Airbus and Rolls Royce are looking into utilizing quantum computers for airplane and turbojet engines.



{ Application 3 — Optimization & machine learning



- Scheduling, routing, portfolios: vast spaces of possible choices.
- Quantum methods may explore certain **landscapes** differently.
- In ML, quantum models could enrich how data is represented.

Reality check: advantages here are **problem-specific** and still being established.

The hybrid split

Classical: train, optimize, decide.

Quantum: propose or evaluate hard candidates in the loop.



{ Quantum computing in Europe — available now



Europe's strategy mirrors this talk: quantum **integrated with** classical HPC.

- **EuroHPC JU** is deploying six quantum systems, each **coupled to a Tier-0 supercomputer** — superconducting, trapped-ion, photonic, and neutral-atom platforms.
- **Superconducting:** Euro-Q-Exa (LRZ, Germany) — 54 qubits now, 150 by end of 2026. IQM (Finland/Germany) builds the hardware.
- **Trapped ion:** AQT (Innsbruck, Austria) — 12-qubit IBEX on the cloud, first quantum hardware for EU customers on AWS Braket.
- **Neutral atom:** SOL (Bologna, Italy) — neutral-atom quantum simulator.
- **Annealing:** D-Wave being coupled to JUPITER (Jülich, Germany), Europe's first exascale machine.



{ Hybrid systems in practice — happening now

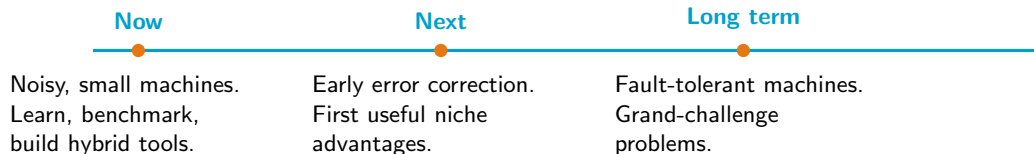


The recurring pattern: the QPU becomes **just another accelerator** in the HPC job queue, next to CPUs and GPUs.

- **Europe (LRZ, Germany):** IQM quantum computers run as a **Slurm node** inside the supercomputer — quantum jobs scheduled alongside CPU/GPU jobs, using the same workload manager HPC already relies on.
- **US national labs:** DOE integrates IBM, Rigetti, and IonQ hardware at Oak Ridge and Argonne; quantum **subroutines embedded inside classical simulation pipelines** (Qiskit Runtime, Braket Hybrid Jobs).
- **Exascale link:** optimization algorithms (QAOA) run in hybrid workflows tied to Frontier, connecting quantum kernels to a classical exascale machine.
- **Programming model:** NVIDIA CUDA-Q lets one program co-target **CPU + GPU + QPU** together.



A realistic roadmap



Progress is incremental — and hybrid systems deliver value at every step.



{ Conclusion



- Classical computing evolved relentlessly — and powers modern life.
- A few problems remain **out of reach** for any classical machine.
- Quantum computing is a **fundamentally different** way to compute — powerful for select problems.
- It is **early**: noise and errors are the central challenge.
- The future is quantum **with** classical — hybrid systems for grand challenges in materials, biology, physics, engineering, and AI.

Working together on the problems that matter most.



{ Disclaimer



Some background materials and images in this presentation are adapted from external sources for educational and illustrative purposes only. No authorship or ownership of third-party materials is claimed. All rights remain with their respective copyright holders.





Thank you
Questions?

